JavaScript + React FAQ (Interview Notes)

Table of Contents

1. Difference between var, let, const
2. Closure with practical example
3. Event Loop in JavaScript
4. Difference between == and ===
5. Hoisting
6. Shallow copy vs deep copy
7. Higher-order functions
8. Synchronous vs asynchronous JavaScript
9. Promise and its states
10. Debouncing and throttling
11. Virtual DOM and React
12. React Hooks
13. useEffect in detail
14. useMemo vs useCallback
15. Lifting state up
16. Controlled vs uncontrolled component
17. React key and importance
18. Context API and usage
19. Custom Hooks in React
20. React re-rendering behavior

Tip: Use the table of contents to jump directly to a question.

SKILLHUB

1. What is the difference between `var`, `let`, and `const`?

English

- `var` is function-scoped and can be re-declared and re-assigned.
- `let` is block-scoped and can be re-assigned, but not re-declared in the same scope.
- `const` is block-scoped and cannot be re-assigned after initialization.

Hindi

- `var` function scope में होता है, और इसे दोबारा declare और re-assign दोनों कर सकते हैं।
- `let` block scope में होता है, इसे re-assign कर सकते हैं, लेकिन same scope में दोबारा declare नहीं कर सकते।
- `const` block scope में होता है, initialization के बाद इसे re-assign नहीं कर सकते।

Marathi

- `var` हा function scope मध्ये असतो, आणि त्याला पुन्हा declare आणि re-assign दोन्ही करता येते.
- `let` हा block scope मध्ये असतो, त्याला re-assign करता येते, पण त्याच scope मध्ये पुन्हा declare करता येत नाही.
- `const` हा block scope मध्ये असतो, initialization नंतर त्याला re-assign करता येत नाही.

Code Example

```
var a = 1;
var a = 2; // allowed

let b = 1;
b = 2; // allowed
// let b = 3; // error

const c = 1;
// c = 2; // error
```

Interviewer May Ask Variations

- Why avoid `var` in modern JavaScript?
- What is block scope vs function scope?
- Can `const` object properties be changed?

Strong Interview Answer

SKILLHUB

" `var` is function-scoped and allows redeclaration, which can cause bugs. `let` and `const` are block-scoped. Use `let` for reassignment and `const` by default for safer, predictable code."

---

2. What is closure? Give a practical example.

English

A closure is when an inner function remembers variables from its outer lexical scope even after the outer function finishes.

Hindi

Closure तब होता है जब inner function, outer lexical scope के variables को याद रखता है, चाहे outer function execute हो चुका हो।

Marathi

Closure म्हणजे inner function ला outer lexical scope मधील variables लक्षात राहतात, जरी outer function पूर्ण झाला तरी.

Code Example

```
function createCounter() {
  let count = 0;
  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

Interviewer May Ask Variations

- What is lexical scope?

SKILLHUB

- How are closures used for private variables?
- Why do closures appear in callbacks and hooks?

Strong Interview Answer

"A closure is when a function retains access to variables from its lexical scope even after the outer function returns. It is useful for private state, function factories, and callbacks."

---

3. Explain the Event Loop in JavaScript.

English

JavaScript is single-threaded. The Event Loop moves callbacks from Web APIs/Task queues to the Call Stack when the stack is empty.

Hindi

JavaScript single-threaded होती है। Event Loop callbacks को queue से Call Stack में तब भेजता है जब stack खाली होता है।

Marathi

JavaScript single-threaded असते. Call Stack रिकामा झाल्यावर Event Loop queue मधील callbacks stack मध्ये पाठवतो.

Code Example

```
console.log("start");
setTimeout(() => console.log("timeout"), 0);
console.log("end");
// start, end, timeout
```

Interviewer May Ask Variations

- What goes to call stack, callback queue, and microtask queue?
- Why do Promises run before `setTimeout`?

SKILLHUB

- Is JavaScript multi-threaded or single-threaded?

Strong Interview Answer

"JavaScript runs on a single call stack. Async operations are handled by browser/Node APIs, and callbacks are queued. The event loop pushes queued tasks to the stack when it is empty; microtasks run before macrotasks."

---

4. What is the difference between `==` and `===`?

English

- `==` checks value after type coercion.
- `===` checks value and type without coercion.

Hindi

- `==` type coercion के बाद compare करता है।
- `===` type और value दोनों को बिना coercion के compare करता है।

Marathi

- `==` type coercion करून compare करतो.
- `===` type आणि value दोन्ही strict compare करतो.

Code Example

```
console.log(5 == "5");  // true
console.log(5 === "5"); // false
```

Interviewer May Ask Variations

- Give examples where `==` returns unexpected true.
- When should we always use strict equality?

SKILLHUB

- What is type coercion?

Strong Interview Answer

"`==` performs type coercion, while `===` compares both type and value strictly. In production code, `===` is preferred to avoid implicit conversion bugs."

---

5. What is hoisting?

English

Hoisting is JavaScript's behavior of moving declarations to the top of their scope before execution.

- `var` is hoisted as `undefined`.
- `let` / `const` are hoisted but stay in Temporal Dead Zone until declaration line.

Hindi

Hoisting JavaScript का behavior है जिसमें declarations execution से पहले scope के top पर process हो जाते हैं।

- `var` `undefined` के साथ hoist होता है, जबकि `let/const` TDZ में रहते हैं जब तक declaration line न आए।

Marathi

Hoisting म्हणजे JavaScript मध्ये declarations execution आधी scope च्या वर process होतात.

- `var` `undefined` सह hoist होतो, तर `let/const` declaration येईपर्यंत TDZ मध्ये राहतात.

Code Example

```
console.log(x); // undefined
var x = 10;

// console.log(y); // ReferenceError
let y = 20;
```

SKILLHUB

Interviewer May Ask Variations

- Are function declarations hoisted?
- What is Temporal Dead Zone?
- Difference between hoisting in `var` and `let/const`?

Strong Interview Answer

"Hoisting means declarations are processed before execution. `var` is hoisted as `undefined`, while `let` and `const` are hoisted but inaccessible in the Temporal Dead Zone until declaration."

---

6. What is the difference between shallow copy and deep copy in JavaScript?

English

- Shallow copy copies top-level properties only; nested objects are shared references.
- Deep copy duplicates nested objects as well.

Hindi

- Shallow copy सिर्फ top-level properties copy करता है; nested objects reference share करते हैं।
- Deep copy nested objects को भी अलग से copy करता है।

Marathi

- Shallow copy फक्त top-level properties copy करतो; nested objects reference share करतात.
- Deep copy nested objects सुद्धा वेगळे copy करतो.

Code Example

```
const original = { user: { name: "A" } };
const shallow = { ...original };
shallow.user.name = "B";
console.log(original.user.name); // B
```

SKILLHUB

```
const deep = structuredClone(original);
deep.user.name = "C";
console.log(original.user.name); // B
```

Interviewer May Ask Variations

- Why is spread operator usually shallow copy?
- How to deep clone nested objects safely?
- JSON clone limitations?

Strong Interview Answer

"Shallow copy duplicates only the first level, so nested references are shared. Deep copy clones nested structures too, preventing accidental mutation of original data."

---

7. What are higher-order functions?

SKILLHUB

English

A higher-order function takes another function as an argument or returns a function.

Hindi

जो function किसी दूसरे function को argument लेता है या function return करता है, उसे higher-order function कहते हैं।

Marathi

जो function दुसरा function argument म्हणून घेतो किंवा function return करतो, त्याला higher-order function म्हणतात.

Code Example

```
const nums = [1, 2, 3];
```

```
const doubled = nums.map(n => n * 2);
```

Interviewer May Ask Variations

- Is `map` a higher-order function?
- Difference between callback and higher-order function?
- Why are HOFs useful in functional programming?

Strong Interview Answer

"Higher-order functions take functions as arguments or return functions. They enable reusable, composable logic and are common in methods like `map`, `filter`, and `reduce`."

---

8. What is the difference between synchronous and asynchronous JavaScript?

SKILLHUB

English

- Synchronous code runs line by line, blocking next operations.
- Asynchronous code allows non-blocking operations (timers, API calls, promises).

Hindi

- Synchronous code line-by-line चलता है और next operation को block करता है।
- Asynchronous code non-blocking होता है (जैसे API calls, timers, promises)।

Marathi

- Synchronous code ओळीनुसार चालतो आणि पुढची कामे block करतो.
- Asynchronous code non-blocking असतो (उदा. API calls, timers, promises).

Code Example

```
// Synchronous
console.log("1");
console.log("2");
console.log("3");
// Output: 1, 2, 3

// Asynchronous
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
// Output: 1, 3, 2
```

Interviewer May Ask Variations

- Does async code run in parallel?
- What blocks the main thread?
- Difference between callback, Promise, and async/await?

Strong Interview Answer

"Synchronous code executes line by line and blocks progress. Asynchronous code delegates long-running tasks and continues execution, improving responsiveness with callbacks, promises, and async/await."

---

9. What is a Promise? Explain its states.

English

A Promise represents a future result of an async operation.

States:

- `pending`
- `fulfilled`
- `rejected`

SKILLHUB

Hindi

Promise async operation के future result को represent करता है।

इसकी states होती हैं:

- `pending`
- `fulfilled`
- `rejected`

Marathi

Promise हा async operation चा भविष्यातील result दर्शवतो.

त्याच्या states:

- `pending`
- `fulfilled`
- `rejected`

Code Example

```
fetch("/api/data")
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Interviewer May Ask Variations

- Can a Promise change state twice?
- Difference between `then`, `catch`, and `finally`?
- How to run multiple Promises together?

Strong Interview Answer

"A Promise represents an async result with states: pending, fulfilled, or rejected. Once settled, state does not change. We handle outcomes using `then`, `catch`, and `finally`."

SKILLHUB

10. What is debouncing and throttling?

English

- Debouncing: run function only after user stops triggering events for a delay.
- Throttling: run function at most once in a fixed interval.

Hindi

- Debouncing: event रुकने के बाद ही function चलाना।
- Throttling: fixed interval में function को सीमित बार चलाना।

Marathi

- Debouncing: events थांबल्यावरच function चालवणे.
- Throttling: ठराविक interval मध्ये function मर्यादित वेळा चालवणे.

Code Example

```
// Debouncing
function debounce(func, delay) {
  let timeout;
  return function(...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => func.apply(this, args), delay);
  };
}

// Throttling
function throttle(func, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      func.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

SKILLHUB

Interviewer May Ask Variations

- Which is better for search input: debounce or throttle?
- How do you implement throttle manually?
- Where is throttle used in UI?

Strong Interview Answer

"Debouncing delays execution until events stop, ideal for search input. Throttling limits execution to fixed intervals, ideal for scroll or resize events."

---

11. What is Virtual DOM? How does React use it?

English

Virtual DOM is an in-memory representation of real DOM. React compares old vs new virtual trees (diffing) and updates only changed real DOM parts.

Hindi

Virtual DOM real DOM का in-memory representation है। React पुराने और नए virtual tree को diff करके सिर्फ changed nodes update करता है।

Marathi

Virtual DOM हा real DOM चा in-memory representation आहे. React जुना आणि नवा virtual tree compare करून फक्त बदललेले nodes update करतो.

Code Example

```
// React automatically handles Virtual DOM diffing
function App() {
  const [count, setCount] = useState(0);
  return (
    
    
    
```

```
        setCount(count + 1)}>Increment

  );
}
```

Interviewer May Ask Variations

- Is Virtual DOM same as real DOM?
- What is reconciliation in React?
- Does React always update full DOM?

Strong Interview Answer

"Virtual DOM is an in-memory representation of UI. React compares previous and current virtual trees and updates only changed real DOM nodes through reconciliation."

---

12. What are React Hooks? Name commonly used hooks.

English

Hooks are functions that let functional components use React features like state and lifecycle.

Common hooks: `useState`, `useEffect`, `useMemo`, `useCallback`, `useRef`, `useContext`, `useReducer`.

Hindi

Hooks ऐसे functions हैं जो functional components को state और lifecycle जैसे React features use करने देते हैं।

Common hooks: `useState`, `useEffect`, `useMemo`, `useCallback`, `useRef`, `useContext`, `useReducer`.

Marathi

Hooks हे असे functions आहेत जे functional components ना state आणि lifecycle सारखी React features वापरू देतात.

Common hooks: `useState`, `useEffect`, `useMemo`, `useCallback`, `useRef`, `useContext`, `useReducer`.

SKILLHUB

Code Example

```
import { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return setCount(count + 1)}>Click me;
}
```

Interviewer May Ask Variations

- Why Hooks cannot be called conditionally?
- Can class components use Hooks?
- When to choose `useReducer` over `useState`?

Strong Interview Answer

"React Hooks let functional components use state and lifecycle-like logic. Common hooks include `useState`, `useEffect`, `useMemo`, `useCallback`, `useRef`, and `useContext`."

---

13. Explain `useEffect` in detail.

English

`useEffect` handles side effects (API calls, subscriptions, timers, DOM updates).

- No dependency array: runs after every render.
- Empty array `[]`: runs once after mount.
- With deps `[a, b]`: runs when any dependency changes.
- Can return cleanup function for unmount or before re-run.

SKILLHUB

Hindi

`useEffect` side effects handle करता है (API calls, subscriptions, timers, DOM updates)।

- Dependency array न हो तो हर render के बाद चलता है।
- `[]` हो तो mount के बाद एक बार चलता है।
- `[deps]` हो तो dependency change होने पर चलता है।
- Cleanup function return करके unmount या rerun से पहले cleanup कर सकते हैं।

Marathi

`useEffect` side effects हाताळतो (API calls, subscriptions, timers, DOM updates).

- Dependency array नसल्यास तो प्रत्येक render नंतर चालतो.
- `[]` असल्यास mount नंतर एकदाच चालतो.
- `[deps]` असल्यास dependency बदलल्यावर चालतो.
- Cleanup function return करून unmount किंवा rerun आधी cleanup करता येतो.

Code Example

```
useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);
  return () => clearInterval(id);
}, []);
```

Interviewer May Ask Variations

- Why does `useEffect` run twice in development Strict Mode?
- How to avoid infinite loops in effects?
- When is cleanup executed?

Strong Interview Answer

"`useEffect` manages side effects such as API calls, subscriptions, and timers. Dependency arrays control execution timing, and cleanup functions prevent leaks on unmount or before reruns."

---

14. What is the difference between `useMemo` and `useCallback`?

SKILLHUB

English

- `useMemo` memoizes a computed value.
- `useCallback` memoizes a function reference.

Hindi

- `useMemo` computed value को memoize करता है।
- `useCallback` function reference को memoize करता है।

Marathi

- `useMemo` computed value memoize करतो.
- `useCallback` function reference memoize करतो.

Code Example

```
const expensiveValue = useMemo(() => heavyCalc(data), [data]);
const onClick = useCallback(() => doSomething(id), [id]);
```

Interviewer May Ask Variations

- When does `useMemo` hurt performance?
- Can `useCallback` prevent child rerenders?
- Are they optimization tools or defaults?

Strong Interview Answer

"`useMemo` memoizes computed values; `useCallback` memoizes function references. Both are performance optimizations and should be used when rerender cost is measurable."

---

15. What is lifting state up in React?

English

SKILLHUB

Moving shared state to the nearest common parent so multiple child components can use and sync the same data.

Hindi

जब multiple child components को same data चाहिए होता है, तो state को nearest common parent में shift करना lifting state up कहलाता है।

Marathi

जेव्हा अनेक child components ना समान data हवा असतो, तेव्हा state nearest common parent कडे नेणे म्हणजे lifting state up.

Code Example

```
// Parent Component
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>
  );
}
```

Interviewer May Ask Variations

- How to share state between sibling components?
- What is prop drilling after lifting state?
- When should Context replace lifted state?

Strong Interview Answer

"Lifting state up means moving shared state to the nearest common parent so sibling components can stay synchronized through props."

---

16. What is controlled vs uncontrolled component?

SKILLHUB

English

- Controlled: form data is managed by React state.
- Uncontrolled: form data is handled by DOM (`ref`).

Hindi

- Controlled component में input value React state control करती है।
- Uncontrolled component में input value DOM/ref से manage होती है।

Marathi

- Controlled component मध्ये input value React state नियंत्रित करते.
- Uncontrolled component मध्ये input value DOM/ref द्वारे manage होते.

Code Example

```
// Controlled
{name}  setName(e.target.value)} />

// Uncontrolled
```

Interviewer May Ask Variations

- Which is easier to validate in forms?
- Why use uncontrolled with file input?
- Tradeoff between controlled and uncontrolled?

Strong Interview Answer

"In controlled components, React state is the source of truth. In uncontrolled components, the DOM manages input state via refs. Controlled inputs are better for validation and dynamic UI behavior."

---

17. What is React key and why is it important?

SKILLHUB

English

`key` uniquely identifies list items so React can efficiently reconcile updates. Stable keys prevent unnecessary re-renders and state bugs.

Hindi

`key` list item को unique identity देता है, जिससे React efficiently reconcile करता है और unnecessary re-renders या state mismatch bugs कम होते हैं।

Marathi

`key` list items ला unique identity देतो, त्यामुळे React efficient reconciliation करतो आणि unnecessary re-renders किंवा state mismatch bugs टाळतो.

Code Example

```
{users.map(user => (

• {user.name}

))}
```

Interviewer May Ask Variations

- Why not use array index as key?
- How do unstable keys cause UI bugs?
- What is a good key source?

Strong Interview Answer

"`key` gives each list element stable identity for reconciliation. Stable keys help React update efficiently and prevent state mismatch bugs during insertions or reordering."

---

18. What is Context API? When should we use it?

SKILLHUB

English

Context API lets you share data (theme, auth, locale) across component tree without prop drilling.

Use it for global-ish read-heavy state, not for all state management needs.

Hindi

Context API prop drilling के बिना component tree में data share करने देता है (जैसे theme, auth, locale)।

इसे global/common read-heavy data के लिए use करें।

Marathi

Context API मुळे prop drilling शिवाय component tree मध्ये data share करता येतो (उदा. theme, auth, locale).

हे global/common read-heavy data साठी वापरावे.

Code Example

```
 // Creating Context
const ThemeContext = createContext();

// Provider
function App() {
  return (


  );
}

// Consumer
function Toolbar() {
  const theme = useContext(ThemeContext);
  return
{theme}
;
}
```

Interviewer May Ask Variations

- Does Context replace Redux?
- How to optimize Context rerenders?

SKILLHUB

- When should you split contexts?

Strong Interview Answer

"Context API shares cross-cutting data like auth or theme without prop drilling. It is best for widely used app-level data, not every local state scenario."

---

19. What are Custom Hooks in React? Why do we use them?

English

Custom Hooks are reusable functions (starting with `use`) that encapsulate stateful logic and side effects.

Used to avoid duplicate logic and improve code readability.

Hindi

Custom Hooks (`use` से शुरू होने वाले) reusable functions होते हैं जो stateful logic और side effects को encapsulate करते हैं।

ये duplication कम करते हैं और code readable बनाते हैं।

Marathi

Custom Hooks (`use` ने सुरू होणारे) reusable functions असतात जे stateful logic आणि side effects encapsulate करतात.

यामुळे duplication कमी होते आणि code readability वाढते.

Code Example

```
function useWindowWidth() {
 const [width, setWidth] = useState(window.innerWidth);

 useEffect(() => {
   const onResize = () => setWidth(window.innerWidth);
   window.addEventListener("resize", onResize);
   return () => window.removeEventListener("resize", onResize);
 }, []);
```

SKILLHUB

```
  return width;
}
```

Interviewer May Ask Variations

- Rules for naming custom hooks?
- Can custom hooks share state between components?
- How do hooks improve testability?

Strong Interview Answer

"Custom hooks are reusable functions that encapsulate stateful logic using built-in hooks. They reduce duplication and keep components cleaner and easier to test."

---

20. How does React handle re-rendering? When does a component re-render?

English

A component re-renders when:

- Its state changes
- Its props change
- Its consumed context value changes
- Parent re-renders (unless optimized with `React.memo`)

React uses reconciliation to compare previous and next virtual DOM, then updates only changed parts in real DOM.

Hindi

React component तब re-render होता है जब:

- State बदलता है
- Props बदलते हैं

SKILLHUB

- Consumed context value बदलता है
- Parent re-render होता है

React reconciliation के through सिर्फ changed DOM parts update करता है।

Marathi

React component re-render होतो जेव्हा:

- State बदलतो
- Props बदलतात
- Consumed context value बदलते
- Parent re-render होतो

React reconciliation वापरून फक्त बदललेले DOM भाग update करतो.

Code Example

```
function Parent() {
  const [count, setCount] = useState(0);
  return (
    <>

      setCount(count + 1)}>Update

  );
}

// Child re-renders when parent re-renders or props change
const Child = React.memo(({ count }) => {
  return
{count}
;
});
```

Interviewer May Ask Variations

- Does parent rerender always rerender child?
- How does `React.memo` help?
- What triggers rerender without state changes?

Strong Interview Answer

SKILLHUB

"React rerenders when state, props, or consumed context change, and often when parents rerender. Reconciliation ensures only actual DOM differences are committed."

Interview Quick Tips

- Prefer `===` over `==`.
- Use `let/const` instead of `var`.
- Explain concepts with one practical example.
- In React answers, mention performance (`memo`, `useMemo`, `useCallback`) only when needed.
- Keep definitions short, then add real-world use.

SKILLHUB